



**Technical Specification Document - Fonepay
Merchant Integration through
Device for Dynamic QR**

**July 2025
Version 1.2**

REVISION HISTORY

Version	Date	Description/Amendment	Created/Modified By
1.0	Aug 2023	First version of the document	Engineer - DevOps & System Integration
1.1	Jul 2024	Updated Test Credentials and API Request	Technical Product Manager
1.2	Jul 2025	Rephrased and restructured content	Technical Product Manager

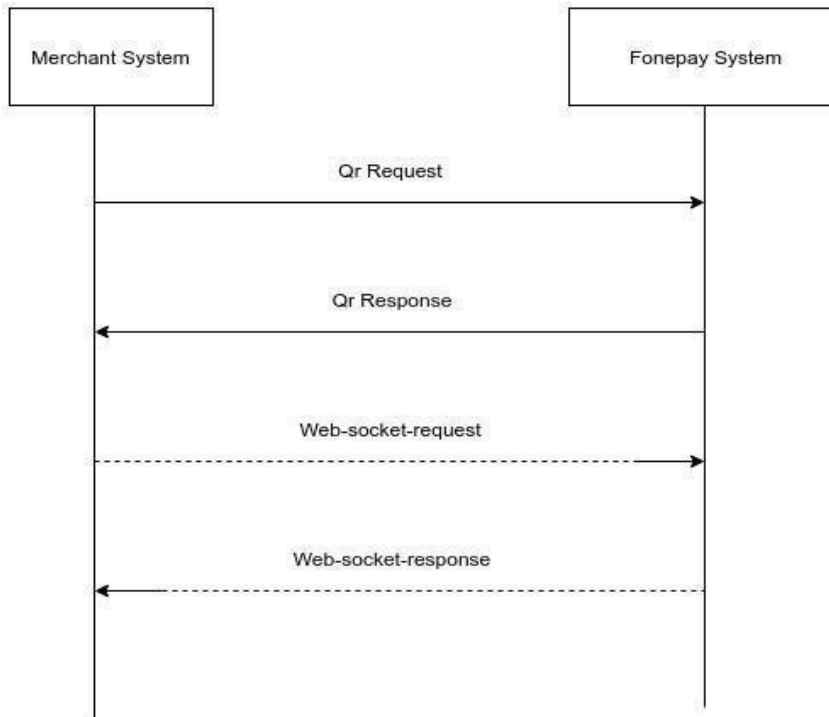
Table of Contents

REVISION HISTORY	1
1. Introduction	2
2. System Flow	2
3. Implementation	2
3.1. QR Request	2
3.1.1. QR Request Parameters.....	2
3.1.2. HMAC Signature for QR request	3
3.2. Web Socket Connection.....	3
3.2.1. WebSocket JSON Message for QR Verification and QR Payment	3
3.3. Check QR Request Status	4
3.3.1. Check QR Request Parameters.....	4
3.3.2. HMAC Signature for Check QR Request	5
3.4. Testing Credentials	5
3.5. Postman Curl.....	5

1. Introduction

This document outlines the standard interface specifications between the Merchant System and the Fonepay System for generating QR data. It provides technical-level details for API-based communication related to QR data. Any data that does not confirm strictly to the specified format will be rejected.

2. System Flow



The Merchant System sends a QR request to the Fonepay System, which then validates the request data and responds accordingly. To receive transaction acknowledgements, the Merchant System must initiate a web-socket connection with the Fonepay Server.

The Fonepay System will use this web-socket connection to send QR verification status and Payment notifications—indicating whether the payment is approved, rejected, or pending—to the Merchant System. **The merchant system must verify the amount and other details in the web-socket response before delivery of service.**

3. Implementation

3.1. QR Request

Merchant needs to send POST request from Merchant System to Fonepay System with following details:

Example:

`https://{Base_URL_API}/api/merchant/merchantDetailsForThirdParty/thirdPartyDynamicQrDownload`

ws://{Base_URL_WS}/merchantEndPoint

3.1.1. QR Request Parameters

Fieldname	Format	Length	Condition	Description
amount	N	1-10	mandatory	The value must consist only of numeric digits "0" to "9" and may include a single "." as the decimal separator. If the amount contains decimal places, the "." should be used to separate the fractional part from the whole number.
remarks1	AN	1-25	mandatory	Narration of the transaction.
remarks2	AN	1-25	mandatory	If additional narration is needed, it can be sent in this field.
prn	ANS	1-25	mandatory	Product Number. This should be unique for all request
merchantCode	AN	1-15	mandatory	Merchant Code provided by Fonepay system
dataValidation	ANS		mandatory	Signature: (HMAC_SHA512Hash)
username	ANS	1-25	mandatory	Username provided by Fonepay system
password	ANS	1-25	mandatory	Password provided by Fonepay system

3.1.2. HMAC Signature for QR request

To generate the Data Validation Token (DV), create a HMAC_SHA512 hash using the Secret Key, which can be found on your Merchant Profile Page after logging in.

Note: Do not share your Secret Key with anyone and avoid storing it in locations that are easily accessible, such as the frontend of a website or app. Always generate the HMAC_SHA512 hash in the backend and securely store the Secret Key there.

3.1.2.1. HMAC For Request

Message for HMAC_SHA512 => {AMOUNT},{PRN},{MERCHANT-CODE},{REMARKS1},{REMARKS2}

Replace the above parameter with the actual value.

Note: Values must be comma-separated and should not be URL-encoded.

For Example:

Key => a7e3512f5032480a83137793cb2021dc

Message => 14,5d76d323-d1f6,NBQM,test1,test2

So, our Data Validation Token (DV) is

5ae718032328ae5615fa4694dfbf3ecd13432bd49031e7285a3f8bc122ecbb8e833e83c7a11f895d30f3e8fd1906317aece113675166ae0d804ecf32a8bbdbaf

3.1.2.2. Sample code

```
public String generateHash(String secretKey, String message) {
```

```

Mac sha512_HMAC = null;

String result = null;
try {
    byte[] byteKey = secretKey.getBytes("UTF-8");
    final String HMAC_SHA512 = "HmacSHA512";

    sha512_HMAC = Mac.getInstance(HMAC_SHA512);

    SecretKeySpec keySpec = new SecretKeySpec(byteKey, HMAC_SHA512);
    sha512_HMAC.init(keySpec);

    result = bytesToHex(sha512_HMAC.doFinal(message.getBytes("UTF-8")));
    return result;

} catch (Exception e) {
    log.error("Exception while Hashing Using HMAC256");
    return null;
}

private static String bytesToHex(byte[] bytes) {
    final char[] hexArray = "0123456789ABCDEF".toCharArray();
    char[] hexChars = new char[bytes.length * 2];

    for (int j = 0; j < bytes.length; j++) {
        int v = bytes[j] & 0xFF;

        hexChars[j * 2] = hexArray[v >>> 4];
        hexChars[j * 2 + 1] = hexArray[v & 0x0F];
    }

    return new String(hexChars);
}

```

3.1.2.3. JSON Message Sample

JSON format to be generated by the Merchant System when sending a QR request, along with the expected response from the Fonepay System, is as follows:

https://{Base_URL_API}/api/merchant/merchantDetailsForThirdParty/thirdPartyDynamicQrDownload

Request

```

{
    "amount": "14",
    "remarks1": "test1",
    "remarks2": "test2",
    "merchantCode": "NBQM",
    "dataValidation":
    "43d2f0939e58e038c3122cc1e65f86af01998dce3e9f70a41a664dc0dbd45dfd74b4c4cbb77af
    ef8a5ae9854ab48fcb d7edfc93156f663a8c60f28830e
    aca7d7",
    "username": "admin",
    "password": "admin123456"
}

```

Success Response

```
{
  "message": "successfull",
  "qrMessage": "000201010212153137910524005204460000000NBQM:29226400011fonepay.
com0104NBQM020329206061367695204541153035245402145802NP5911Fonetest6008Distri
ct622107032 92021098418456336304d3f7",
  "status": "CREATED",
  "statusCode": 201,
  "success": true,
  "thirdpartyQrWebSocketUrl":
"wss://{Base_URL_WS}/merchantEndPoint/Td35588c2d9a647f28f4959f96f905bec/NBQM/Y"
}
```

Failure Responses

```
{
  "documentation": "Merchant request Exception",
  "errorCode": 406,
  "message": "Data Validation Failed"
}
```

```
{
  "documentation": "Merchant request Exception",
  "errorCode": 406,
  "message": "Invalid Merchant Code"
}
```

```
{
  "documentation": "Merchant request Exception",
  "errorCode": 406,
  "message": "Invalid username"
}
```

3.2. Web Socket Connection

A WebSocket connection can be established between the server and browser to receive QR verification notifications and Payment status updates. Once a message is received via WebSocket, the Merchant System should invoke the Check QR Request Status API.

For implementing WebSocket, the Merchant System must use the thirdpartyQRWebSocketUrl provided in the JSON response during the Merchant QR request to initiate the connection.

```
ws://{Base_URL_WS}/merchantEndPoint/556780a5-1239-4a79-a2b4-
48e0cbf1de13/4271420000054621/Y
```

3.2.1. WebSocket JSON Message for QR Verification and QR Payment

When the customer scans the QR and completes the payment, the server will send the QR verification and payment request as outlined below.

QR Verification Response	<pre>{ "merchantId": 70,</pre>
--------------------------	----------------------------------

	<pre> "deviceId": "Td35588c2d9a647f28f4959f96f905bec", "transactionStatus": "{ \"success\": true, \"message\": \"VERIFIED\", \"QRVerified\":true }" } </pre>
QR Payment Response	<pre> Payment Success: { "merchantId": 70, "deviceId": "Td35588c2d9a647f28f4959f96f905bec", "transactionStatus": "{ \"remarks1\": \"test1\", \"remarks2\": \"test2\", \" transactionDate\": \"Apr 11, 2019 1:43:23 PM\", \"productNumber\": \"5 d76d323-d1f6-4a38-8231-0063f9581c98\", \"amount\": \"14\", \" message\": \"Request Complete\", \"success\": true, \" commissionType\": \"CHARGE\", \"commissionAmount\": 0.0, \" </pre>

	<pre>totalCalculatedAmount\": 14.0, \"paymentSuccess\": true } }</pre>
--	--

Note

Check paymentSuccess of transactionStatus from WebSocket response

```
{
  "traceld":17015,
  "remarks1":"test1",
  "remarks2":"test2",
  "transactionDate":"Apr 11, 2019 1:43:23 PM",
  "productNumber": "5d76d323-d1f6-4a38-8231- 0063f9581c98",
  "amount": "14",
  "message": "Request Complete",
  "success": true,
  "commissionType": "CHARGE",
  "commissionAmount": 0.0,
  "totalCalculatedAmount": 14.0,
  "paymentSuccess": true
}
```

3.3. Check QR Request Status

The Merchant system should verify the QR status upon receiving the WebSocket notification. If no response is received via the WebSocket, Merchant can alternatively check the QR request status.

To do so, Merchant needs to send a POST request with the following details:

Example: https://{Base_URL_API}/api/merchant/merchantDetailsForThirdParty/thirdPartyDynamicQRGetStatus

3.3.1. Check QR Request Parameters

Field Name	Format	Length	Condition	Description
prn	ANS	1-25	mandatory	Product Number. prn send during QR request
merchantCode	AN	1-15	mandatory	Merchant Code provided by Fonepay system
dataValidation	ANS		mandatory	Signature (HMAC_SHA512 Hash)
username	ANS	1-25	mandatory	Username provided by Fonepay system
password	ANS	1-25	mandatory	Password provided by Fonepay system

3.3.2. HMAC Signature for Check QR Request

To generate the Data Validation Token (DV), create a hash (H) using the Secret Key, which can be found on your merchant profile page after logging in.

Note: Do not share your Secret Key with anyone and avoid storing it in locations that are easily accessible, such as the frontend of a website or app. Always generate the hash in the backend and securely store the Secret Key there.

3.3.2.1. HMAC For Request

Message for HMAC_SHA512 => {PRN},{MERCHANT-CODE}

Replace Param above with actual value.

Note: Value is separated by comma and value should not be URL encoded.

For Example:

Key=> a7e3512f5032480a83137793cb2021dc

Message => 5d76d323-d1f6-4a388,NBQM

So, our Data Validation Token (DV) is

b816affc4599162bdbd9b8c7f6b7b83508dadfcceb0d58ba01f1042d749a7b5d71ea5b6f409ca2d61607043555733c9240d2651a3473e7a195a326b21e9fafe8

3.3.2.2. JSON Message Sample

JSON format to be generated by the Merchant System when sending a Check QR request, along with the expected response from the Fonepay System, is as follows:

https://{Base_URL_API}/api/merchant/merchantDetailsForThirdParty/thirdPartyDynamicQrGetStatus

Request

```
{
  "prn": "5d76d323-d1f6",
  "merchantCode": "NBQM",
  "dataValidation":
  "de5fd3bbbd7d36c766a47c0a137e41de7587028d2f6e3deacb5bebe30992326876a6fba4f9ccfd55a1d302a81aba94733d6c1db04f749483be63b619a9b032b7",
  "username": "admin",
  "password": "admin123456"
}
```

Success Response

```
{
  "fonepayTraceId": 17404,
  "merchantCode": "NBQM",
  "paymentStatus": "success",
  "prn": "5d76d323-d1f6"
}
```

Failed Response

```
{
  "fonepayTraceId": 17420,
  "merchantCode": "NBQM",
  "paymentStatus": "failed",
  "prn": "654df0eb-0740"
}
```

Pending Response

```
{
  "fonepayTraceId": 17420,
  "merchantCode": "NBQM",
  "paymentStatus": "failed",
  "prn": "654df0eb-0740"
}
```

3.4. Testing Credentials

merchant code: fonepay123
secret key: fonepay

username: bijayk
password: password

3.5. Postman Curl

```
curl -location 'https://
{Base_URL_API}/api/merchant/merchantDetailsForThirdParty/thirdPartyDynamicQrDownload' \
--header 'Content-Type: application/json' \
--data '{
  "amount": "50",
  "prn": "check",
  "merchantCode": "fonepay123",
  "remarks1": "test1",
  "remarks2": "test2",
  "dataValidation":
"f036eb09c5402a91e1926e904a423e66d041add18423959dad512b6f5fa37f1ea023197f6faae4
2f84b357914f5ed77723255d108b38c20c058677cfc8ac4204",
  "username": "bijayk",
  "password": "password"
}'
```

*****End of Document*****